Assignment No:- 04

Title:- Design n-Queens matrix having first Queen placed. Use backtracking to place remaining Queens to generate the final n-queen's matrix.

Program Code:-

```
class NQueens:
    def __init__(self, N=4):
        self.N = N

    def printSolution(self, board):
        for i in range(self.N):
            for j in range(self.N):
                print(board[i][j], end=" ")
            print()

    def isSafe(self, board, row, col):
        # Check this row on left side
        for i in range(col):
            if board[row][i] == 1:
                return False

        # Check upper diagonal on left side
        i, j = row, col
        while i >= 0 and j >= 0:
            if board[i][j] == 1:
                return False
            i -= 1
            j -= 1

        # Check lower diagonal on left side
        i, j = row, col
        while i < self.N and j >= 0:
            if board[i][j] == 1:
                return False
            i += 1
            j -= 1

        return True

    def solveNQUtil(self, board, col):
        if col >= self.N:
            return True

        for i in range(self.N):
            if self.isSafe(board, i, col):
                board[i][col] = 1

                if self.solveNQUtil(board, col + 1):
                    return True

                board[i][col] = 0  # BACKTRACK

        return False

    def solveNQ(self):
        board = [[0 for _ in range(self.N)] for _ in range(self.N)]
```

```
        if not self.solveNQUtil(board, 0):
            print("Solution does not exist")
            return False

        self.printSolution(board)
        return True


if __name__ == "__main__":
    Queen = NQueens(4)
    Queen.solveNQ()
```

OUTPUT:-

```
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0
```